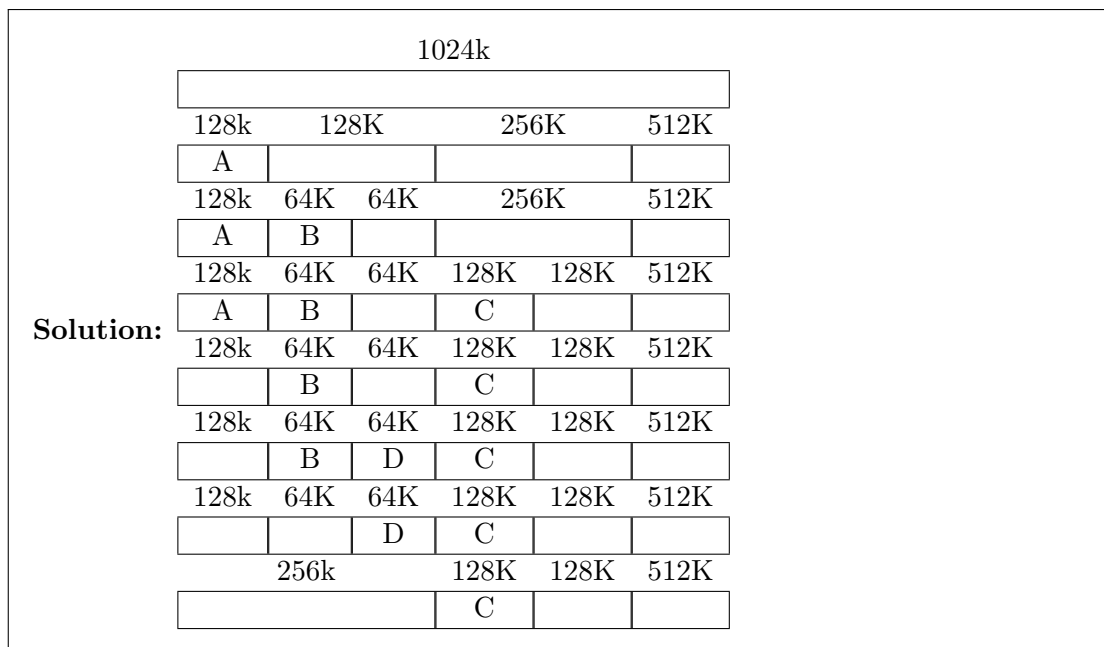


Geheugen Management

1 Buddy systeem

1. Beschouw een buddy systeem van 1Mbyte groot. Geef grafisch de toestand van het geheugen weer na volgende acties, als je weet dat het geheugen initieel leeg is.

- (a) Request A = 70 Kbyte
- (b) Request B = 35 Kbyte
- (c) Request C = 80 Kbyte
- (d) Return A
- (e) Request D = 60 Kbyte
- (f) Return B
- (g) Return D



2. In het standaard buddy systeem krijgen de processen uiteindelijk geheugenblokken toegewezen van machten van 2. Stel dat we in plaats daarvan een Fibonacci reeks willen gebruiken.

- Leg de werking van zo een systeem uit aan de hand van een kort voorbeeld. (tip: veronderstel dat de totale geheugengrootte een (groot) Fibonacci getal is)

Solution: Beschouw een buddy systeem van 89KB groot.

- (a) Request X = 12 Kbyte

(b) Request Y = 12 Kbyte

(c) Return X

(d) Request Z = 4 Kbyte

89k				
13k	21K		55K	
X				
13k	8K	13K	55K	
X			Y	
13k	8K	13K	55K	
			Y	
13k	3K	5K	13K	55K
		Z	Y	

- Presteert dit systeem beter of slechter dan het standaard buddy systeem qua interne fragmentatie ? Waarom (niet) ?

Solution: We veronderstellen een uniforme verdeling van de requestgroottes (tussen bepaalde, niet gespecificeerde grenzen, maar groot genoeg). De n-de term in de Fibonacci reeks noemen we f_n , en we weten dat

$$\lim_{n \rightarrow \infty} \frac{f_{n+1}}{f_n} = \frac{1 + \sqrt{5}}{2} = \phi.$$

- Standaard buddy systeem: elk bezet stuk geheugen is minstens voor 1/2 gevuld, en hoogstens volledig gevuld. Gemiddeld: 3/4 gevuld. Voor een aangevraagd stuk geheugen wordt er dus gemiddeld $4/3 - 1 \cong 33\%$ extra geheugen gealloceerd.
- Fibonacci buddy systeem: elk bezet stuk geheugen is minstens voor f_n/f_{n+1} gevuld, en hoogstens volledig gevuld. Gemiddeld: $\frac{1}{2} \frac{2}{1+\sqrt{5}} + \frac{1}{2} 1 \cong 81\%$ gevuld. Voor een aangevraagd stuk geheugen wordt er dus gemiddeld $(81\%)^{-1} - 1 \cong 23\%$ extra geheugen gealloceerd.

Hints i.v.m. Fibonacci:

$$\phi = \frac{1 + \sqrt{5}}{2} \quad \psi = \frac{1 - \sqrt{5}}{2} = 1 - \phi$$

$$f_n = \frac{\phi^n - \psi^n}{\phi - \psi} = \frac{\phi^n - \psi^n}{\sqrt{5}}$$

$$\phi^2 = \phi + 1, \phi^{n+1} = \phi^n + \phi^{n-1}$$

$$f_{n+1} = f_n + f_{n-1} \Rightarrow \frac{f_{n+1}}{f_n} = \frac{f_n}{f_n} + \frac{f_{n-1}}{f_n} \Rightarrow \phi = 1 + \frac{1}{\phi}$$

Paging

1 Omvang van de page tables van een proces

Stel dat we een 32-bit machine hebben met virtuele adressen die zijn opgesplitst in 4 delen van 10, 8, 6 en 8 bits respectievelijk. We hanteren deze adressen in een 3 level page table systeem, waarbij de eerste 10 bits gebruikt worden voor level 1, de volgende 8 bits voor level 2, etc. Een enkele page table entry is 4 bytes groot. Het systeem is byte geadresseerd.

1. Wat is de page size in dit systeem?

Solution: 2^8 byte

2. Stel dat een process 256K geheugen vereist (dat start vanaf adres 0), hoeveel plaats nemen de page tables van dit proces dan in beslag?

Solution:

- 1 page = 2^8 bytes
- 1 L_3 table = 2^6 pages
- 1 L_2 table = 2^8 L_3 tables
- $256KB = 2^{18}B$ dus 2^{10} pages, 2^4 L_3 page tables, 1 L_2 page tables en 1 L_1 page table.
- Totaal: $4 \times (2^4 \times 2^6 + 1 \times 2^8 + 1 \times 2^{10}) = 9216B$

3. Stel dat een proces een code segment heeft van 48K vanaf (virtueel) adres 0x10000000, een 600K data segment vanaf adres 0x80000000 en een stack segment van 64K vanaf adres 0xF0000000. Hoeveel plaats vereist de page table van dit proces?

Solution: $0x1000\ 0000 = 2^{28}B$

- 48KB dus 192 pages, 3 L_3 tables, 1 L_2 tables en 1 L_1 table.
- 600KB dus 2400 pages, 38 L_3 tables, 1 L_2 tables en 1 L_1 table.
- 64KB dus 256 pages, 4 L_3 tables, 1 L_2 tables en 1 L_1 table.
- Controleer op samenvallende tables, op basis van hun adres.
 - 1 L_3 table "covert" $2^6 2^8 = 2^{14}B = 0x4\ 0000$

- 1 L_2 table "covert" $2^8 2^6 2^8 = 2^{22}B = 0x400\ 0000$
- 1 L_2 table "covert" $2^{10} 2^8 2^6 2^8 = 2^{22}B = 0x1\ 0000\ 0000$

In dit geval vallen alle L_1 tables samen

- Totaal: $4 \times (45 \times 2^6 + 3 \times 2^8 + 1 \times 2^{10}) = 18688B$

2 Bepaal het aantal levels in een multilevel paging systeem

Stel dat een machine virtuele adressen heeft bestaande uit 36 bits en dat de pages 8K groot zijn. Een enkele page entry is 8 bytes lang.

1. Uit hoeveel paginas bestaat de virtuele adresruimte?

Solution: $\frac{2^{36}}{8K} = \frac{2^{36}}{2^{13}} = 2^{23}$ pages

2. Stel dat een gemiddeld proces 8 Gbyte groot is. Hoeveel levels zijn er dan het meest geschikt indien we met een multilevel page table werken (bekijk 1 tot 3 levels)? Weeg de voor- en nadelen tegen elkaar af. Voor het systeem met 2 levels kies je de omvang van een level 2 page table zodanig dat deze precies past in een enkele page. Voor drie levels neem je een min of meer gelijke opdeling voor elk van de velden.

Solution:

- Single level:

L1	offset
23	13

 - $2^{23} \text{PTEs} \times 8 \frac{\text{byte}}{\text{PTE}} = 2^{26} \text{byte} = 64MB$
- 2 levels:

L1	L2	offset
13	10	13

 - 1 page = 2^{13} bytes (kan 2^{10} page table entries bevatten)
 - 1 L_2 table = 2^{10} pages
 - $8GB = 2^{33}B$ dus 2^{20} pages, 2^{10} L_2 pages en 1 L_1 page.
 - Totaal: $8 \times (2^{10} \times 2^{10} + 1 \times 2^{13}) = 8256KB$
- 3 levels:

L1	L2	L3	offset
7	8	8	13

 - 1 page = 2^{13} bytes
 - 1 L_3 table = 2^8 pages

- $1L_2$ table = $2^8 L_3$ tables
- $8GB = 2^{33}B$ dus 2^{20} pages, $2^{12} L_3$ pages, $2^4 L_2$ pages en 1 L_1 page.
- Totaal: $8 \times (2^{12} \times 2^8 + 2^4 \times 2^8 + 1 \times 2^7) = 8225KB$

Het systeem met 1 level vereist duidelijk meer geheugen dan 2 of 3 levels. Tussen deze twee laatste is niet veel verschil, maar een extra level kost meer, dus 2 levels lijkt het interessants.

3 Tel het aantal TLB misses

Beschouw volgend stukje code. Stel dat de binaire code op één page past en dat de stack eveneens op een enkele page past. Je mag er bovendien van uit gaan dat bij het aanroepen van de functie, de stack en code page reeds in de TLB cache zitten. Veronderstel verder dat een integer 4 bytes lang is en dat een page 4096 bytes bevat.

```
int a[1024][1024], b[1024][1024], c[1024][1024];
multiply() {
    unsigned i,j,k;
    for(i=0; i< 1024; i++)
        for(j=0; j< 1024; j++)
            for(k=0; k< 1024; k++)
                c[i][j]+=a[i][k]*b[k][j];
}
```

Gegeven dat de TLB cache slechts 8 entries bevat en dat hij een LRU replacement policy gebruikt, wat is dan het aantal TLB misses bij de uitvoering van deze code?

Solution:

- De pages voor stack en code zitten steeds in TLB, dus blijven er nog 6 TLB entries over.
- Elke array bestaat uit 1024 pages:
 - Page 0 van a bevat $a[0][0]$ tot $a[0][1023]$
 - Page 1 van a bevat $a[1][0]$ tot $a[1][1023]$
 - ...
- Stel i (of alleszins i vast)
 - Requests: $a[i][0], b[0][0], c[i][0], a[i][1], b[1][0], c[i][0], \dots$
 - Deze blijven binnen een enkele page van a en c.
 - 1 miss voor a, 1 miss voor c
 - b: telkens andere page: 1024×1024 misses.
 - Totaal $2 + 1024 \times 1024$
- Dit herhalen voor alle i : $1024 \times (2 + 1024 \times 1024) = 1073743872$.

- $1L_2$ table = $2^8 L_3$ tables
- $8GB = 2^{33}B$ dus 2^{20} pages, $2^{12} L_3$ pages, $2^4 L_2$ pages en 1 L_1 page.
- Totaal: $8 \times (2^{12} \times 2^8 + 2^4 \times 2^8 + 1 \times 2^7) = 8225KB$

Het systeem met 1 level vereist duidelijk meer geheugen dan 2 of 3 levels. Tussen deze twee laatste is niet veel verschil, maar een extra level kost meer, dus 2 levels lijkt het interessants.

3 Tel het aantal TLB misses

Beschouw volgend stukje code. Stel dat de binaire code op één page past en dat de stack eveneens op een enkele page past. Je mag er bovendien van uit gaan dat bij het aanroepen van de functie, de stack en code page reeds in de TLB cache zitten. Veronderstel verder dat een integer 4 bytes lang is en dat een page 4096 bytes bevat.

```
int a[1024][1024], b[1024][1024], c[1024][1024];
multiply() {
    unsigned i,j,k;
    for(i=0; i< 1024; i++)
        for(j=0; j< 1024; j++)
            for(k=0; k< 1024; k++)
                c[i][j]+=a[i][k]*b[k][j];
}
```

Gegeven dat de TLB cache slechts 8 entries bevat en dat hij een LRU replacement policy gebruikt, wat is dan het aantal TLB misses bij de uitvoering van deze code?

Solution:

- De pages voor stack en code zitten steeds in TLB, dus blijven er nog 6 TLB entries over.
- Elke array bestaat uit 1024 pages:
 - Page 0 van a bevat $a[0][0]$ tot $a[0][1023]$
 - Page 1 van a bevat $a[1][0]$ tot $a[1][1023]$
 - ...
- Stel i (of alleszins i vast)
 - Requests: $a[i][0], b[0][0], c[i][0], a[i][1], b[1][0], c[i][0], \dots$
 - Deze blijven binnen een enkele page van a en c.
 - 1 miss voor a, 1 miss voor c
 - b: telkens andere page: 1024×1024 misses.
 - Totaal $2 + 1024 \times 1024$
- Dit herhalen voor alle i : $1024 \times (2 + 1024 \times 1024) = 1073743872$.

4 Demand paging: goed of slecht

Welke van onderstaande programmeer technieken werkt *goed* in combinatie met een demand paged systeem.

1. Stack
2. Hash table
3. Sequentiële zoekmethode
4. Binaire zoekmethode
5. Pure code
6. Vector operaties (bijvoorbeeld vectorvermenigvuldiging)
7. Indirecte adressen

Solution:

1. Stack: +
2. Hash table: -
3. Sequentiële zoekmethode: +
4. Binaire zoekmethode: - (initieel slecht, later beter)
5. Pure code: + (als er weinige “verre” function calls zijn)
6. Vector operaties (bijvoorbeeld vectorvermenigvuldiging): +
7. Indirecte adressen/pointers: typisch - (maar + als pointers dicht bij elkaar liggen)

5 Nuttige ingrepen voor demand paging

Stel dat we enkele statistieken nemen op een demand paged systeem. We merken dat de CPU 20% gebruikt wordt, de disk met de pages 97.3% en de overige I/O devices slechts 5%. Welke ingrepen lijken je doeltreffend om het systeem te verbeteren?

1. Installeer een snellere CPU
2. Installeer een grotere paging disk
3. Verhoog de mate van multiprogrammeren
4. Verlaag de mate van multiprogrammeren
5. Installeer meer geheugen
6. Installeer een snellere hard disk

7. Laat prepaging toe
8. Verhoog de page size

Solution:

1. Installeer een snellere CPU: weinig effect
2. Installeer een grotere paging disk: weinig effect
3. Verhoog de mate van multiprogrammeren: typisch nog slechter (meer processen, dus meer geheugen nodig)
4. Verlaag de mate van multiprogrammeren: typisch beter
5. Installeer meer geheugen: beter
6. Installeer een snellere hard disk: kleine verbetering
7. Laat prepaging toe: afhankelijk van de situatie
8. Verhoog de page size: afhankelijk van de situatie

6 Inverted versus standaard page tables

Indien virtuele adressen 64 bits lang zijn, de page size 1Kbyte is, de omvang van het fysieke geheugen is 64Mbyte en een PTE neemt twee bytes in beslag, wat is dan de totale grootte van de inverted en standaard page table (indien elke byte geadresseerd wordt)? Maak hiervoor eerst een schatting van het aantal bytes een inverted page table entry nodig heeft, door na te gaan welke velden er minstens in moeten zitten, en hoeveel bits ze elk moeten bevatten.

Solution:

- Standard page table:
 - $64\text{bit} : 2^{64}$ byte
 - $\frac{2^{64}}{1K} = 2^{54}$ entries
 - $2^{54} \times 2 \text{ bytes} = 32768 \text{ TByte}$
- Inverted page table:
 - $\frac{64M}{1K} = \frac{2^{26}}{2^{10}} = 2^{16} = 65536$ inverted page table entries
 - Hoeveel byte voor een inverted page table entry: 10 bytes, 80 bit
 - * Virtual page: 2^{54} entries, dus 54 bit.
 - * Pointer naar andere entry: 2^{16} entries, dus 16 bits
 - * Process ID en flags: bvb 10 bit (arbitrair gekozen)
 - $65536 \times 10 = 640K\text{byte}$.

7 Inverted page table

Beschouw een systeem met 16KB geheugen, pages van 1KB groot, een inverted page table en een full-associative TLB met 2 enties en een LRU replacement policy. De gebruikte hash functie is $f(PID, VPN) = (10 * PID + VPN) \bmod 24$. Het systeem heeft 0,5ns nodig om de TLB uit te lezen, 100ns om het main memory uit te lezen en 10ms om de harde schijf raad te plegen. Vul de hieronder gegeven tabel met page requests verder aan gebruik makend van de gegeven hash anchor table en de inverted page table (IPT). In de IPT wordt “T” gebruikt als symbool om aan te duiden dat er geen volgende page table entry (PTE) is.

In de TLB kolom geef je de inhoud van de TLB weer nadat de vertaling gebeurd is. Maak hiervoor gebruik van de notatie “(VPN, Frame number)” voor elke lijn in de cache, gebruik “-” als de desbetreffende lijn leeg is. In de kolommen genaamd “Frame number”, “TLB hits” en “Page fault” geef je respectievelijk de physical frame number weer, of er tijdens de vertaling een TLB hit was en of het aanleiding gaf tot een page fault of niet. Ten slotte vul je in de laatste kolom de tijd in die nodig was om de vertaling uit te voeren. We veronderstellen dat de buffer in parallel geupdate kan worden na het vertalen en het uitvoeren van de volgende instructie, hiervoor hoeft je dus geen tijd te rekenen. In het geval van eventuele page faults hoeft je de datastructuren niet aan te passen, wel dien je de tijd te geven die nodig was om erachter te komen dat er een page fault is.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	7	3	12	15	13	6	2	4	0	14	8	5	15	11	10	8	5	7	9	10	12	6	13	1

Tabel 1: Hash anchor table

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PID	0	3	2	3	0	8	0	2	2	0	8	6	0	2	2	4
VPN	8	17	8	4	1	3	5	4	2	3	13	1	2	32	0	4
Next PTE	2	T	T	4	T	T	10	12	3	T	0	9	14	8	T	11

Tabel 2: Inverted Page table

PID	VPN	TLB	Frame number	TLB hits	Page fault	Latency (ns)
0	1			Hit/Miss	Yes/No	
0	2			Hit/Miss	Yes/No	
0	1			Hit/Miss	Yes/No	
0	3			Hit/Miss	Yes/No	
8	3			Hit/Miss	Yes/No	
0	3			Hit/Miss	Yes/No	
2	2			Hit/Miss	Yes/No	
2	1			Hit/Miss	Yes/No	
8	13			Hit/Miss	Yes/No	
8	3			Hit/Miss	Yes/No	

Solution: De "index"-rij van de inverted page table geeft het page frame aan; de inverted page table entries beschrijven het proces en VPN binnen dat proces.

Enkel tijd voor page fault **detectie** (dus tijd voor TLB check en bij TLB cache miss extra tijd voor hash table en # IPT accesses in main memory); hiervoor is geen access naar harde schijf nodig.

Proceswissel flusht de TLB cache (vandaar dat TLB enkel de vertaling VPN → framennr. bij moet houden).

f	PID	VPN	TLB	Frame number	TLB hits	Page fault	Latency (ns)
1	0	1	(1,4) -	4	Hit /Miss	Yes /No	300,5
2	0	2	(1,4) (2,12)	12	Hit /Miss	Yes /No	200,5
1	0	1	(1,4) (2,12)	4	Hit/ Miss	Yes /No	0,5
3	0	3	(1,4) (3,9)	9	Hit /Miss	Yes /No	400,5
11	8	3	(3,5) -	5	Hit /Miss	Yes /No	200,5
3	0	3	(3,9) -	9	Hit /Miss	Yes /No	400,5
22	2	2	(2,8) -	8	Hit /Miss	Yes /No	300,5
21	2	1	(2,8) -	/	Hit /Miss	Yes / No	500,5
21	8	13	(13,10) -	10	Hit /Miss	Yes /No	300,5
11	8	3	(13,10) (3,5)	5	Hit /Miss	Yes /No	200,5

8 Tel het aantal page faults

Beschouw volgende sequentie van paginas:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

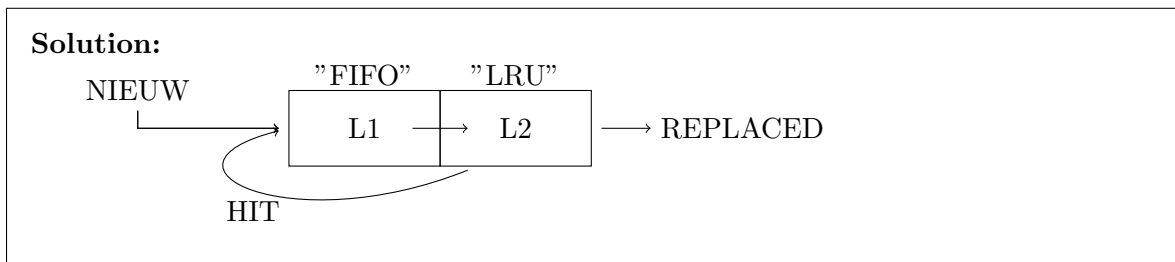
Hoeveel page faults treden er op indien dit process over 1, 2, ..., 6 of 7 frames kan beschikken. Ga dit na voor volgende drie replacement algoritmen: LRU, FIFO en het MIN algoritme. Initieel zijn er geen paginas in het geheugen aanwezig.

Solution: MIN replacet de page die het verste in de toekomst wordt opgevraagd.

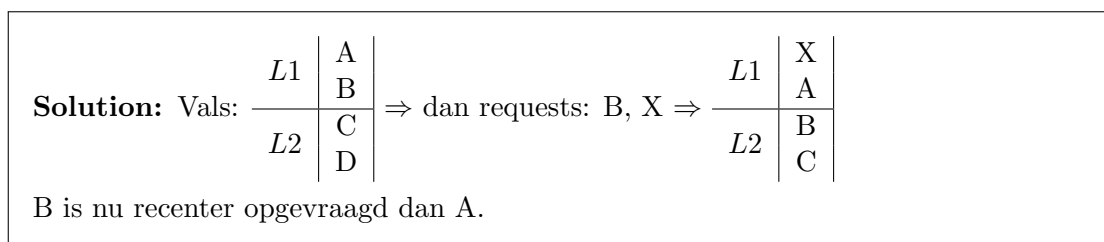
Frames	LRU	FIFO	MIN
1	20	20	20
2	18	18	15
3	15	16	11
4	10	14	8
5	8	10	7
6	7	10	7
7	7	7	7

9 Segmented FIFO

Waar of vals?



1. Voor alle pages in L1 geldt dat ze recenter opgevraagd zijn dan deze in L2.



2. Pages in L1 zitten in LRU volgorde.

Solution: Vals: zitten in FIFO volgorde

3. Pages in L2 zitten in LRU volgorde.

Solution: Vals: zitten in FIFO volgorde

4. $|L2| - 1$ meest recent gebruikte pages zijn safe (kunnen niet vervangen worden).

Solution: Juist, want deze zijn recenter “voor” L_2 geplaatst en kunnen dus nog minstens $|L_2|$ replacements overleven.

5. $|L_2|$ meest recent gebruikte pages zijn safe.

Solution: Juist, want deze zijn recenter “voor” L_2 geplaatst en kunnen dus nog minstens $|L_2|$ replacements overleven.

6. Een segmented FIFO systeem met $|L_1| = |L_2| = 2$ is equivalent aan een tree-based pseudo-LRU systeem met 4 pages.

Solution: Vals, zie oefening 4 bij caching: bij tree-based kan 1 page “beschermd” worden door een buur. Bij segmented FIFO kan dit niet.

10 Benchmarking

Teken een grafiek die een cache benchmark, zoals beschreven op p31 van de theorie cursus, illustreert. We beschouwen een systeem dat een L1 cache en een TLB cache heeft. De grootte van de L1 cache omvat 256 KByte, een cache lijn bevat 32 bytes en er wordt direct-mapping verondersteld. Een L1 cache access neemt 250ns in beslag, een L1 cache miss zorgt voor 500ns extra, een TLB miss nog eens 1000ns. De TLB cache is 4-way associatief, bevat 128 entries, en de page size is 8192 bytes.

Maak de grafiek, met op de x-as de stride-grootte (logaritmische schaal) en de access tijd op de y-as, voor arraygroottes van 4 Kbyte tot 4096 Kbyte.

Solution:

